Lab 7: Fuzzing for Input Validation Bugs

Course: Software Engineering & Testing | **Tool:** Hypothesis Fuzzing Framework
Application: train_fare.py | **Objective:** Uncover edge-case crashes & defensively harden code

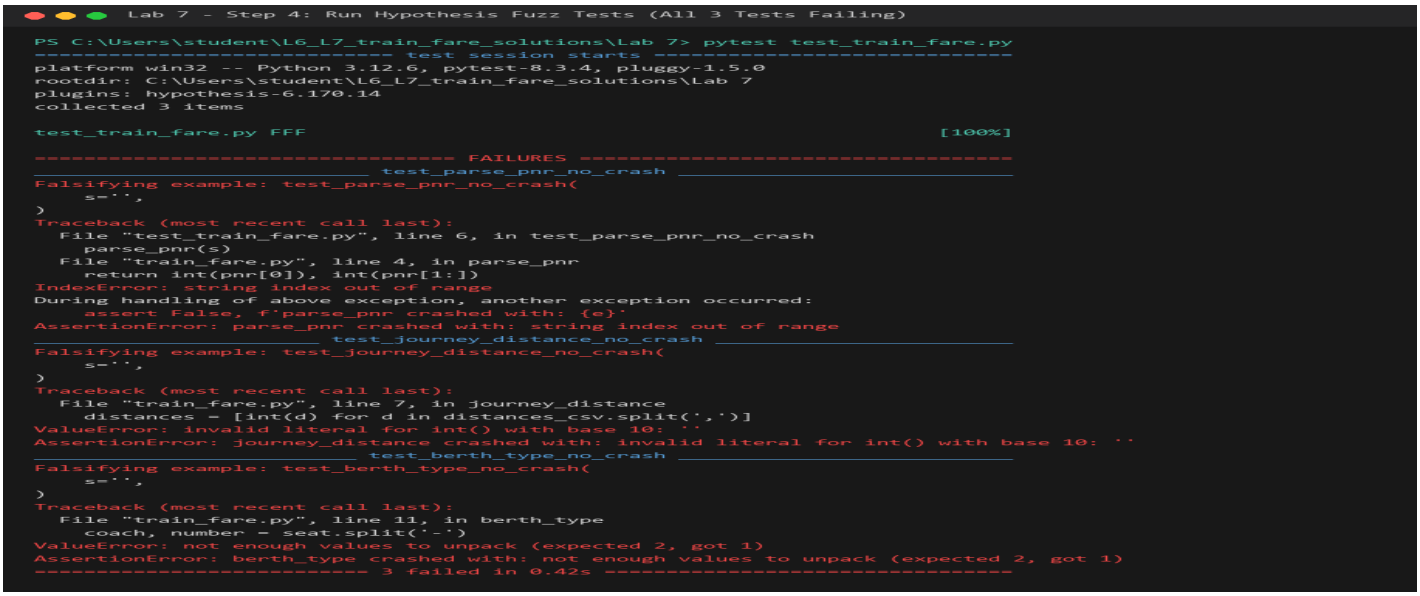
1. Fuzz Testing Methodology & Bugs Discovered

Using Hypothesis property-based testing (@given(st.text() | st.none())), we fuzzed the input parameters of parse_pnr, journey_distance, and berth_type. Hypothesis rapidly generated edge-case inputs that caused unhandled crashes:

- **parse_pnr(pnr):** Crashed with TypeError when pnr=None, IndexError when pnr='', and ValueError on non-digits or length < 2.
- **journey_distance(distances_csv):** Crashed with AttributeError when distances_csv=None, ValueError on empty string '' or non-numeric tokens.
- **berth_type(seat):** Crashed with AttributeError on None, ValueError ('not enough values to unpack') when missing hyphens, and ValueError on invalid berth numbers.

2. Step 2 & Step 4: Buggy Execution & Hypothesis Failure Output

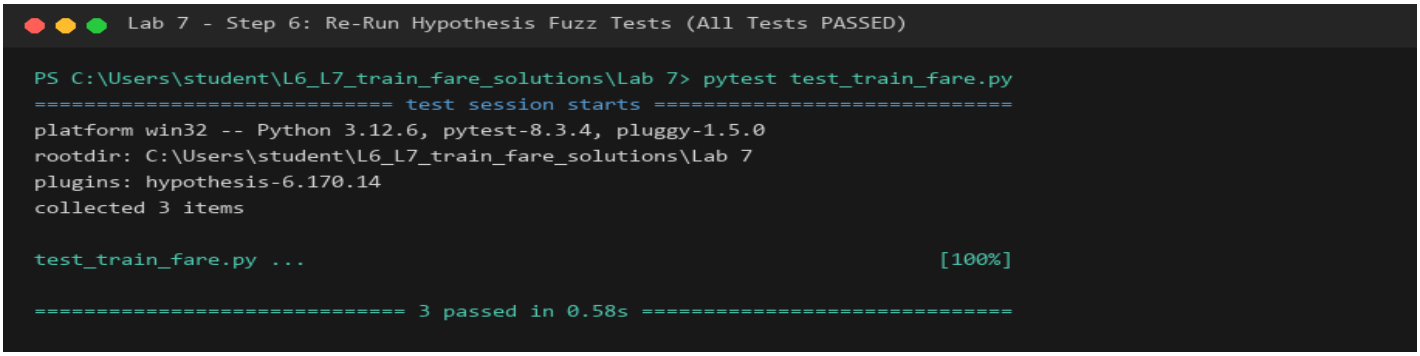
Running the initial buggy code crashed on standard unexpected inputs. When executed under pytest with Hypothesis, all 3 property-based fuzz tests immediately failed with minimal falsifying examples (empty string '', None):



3. Step 5 & Step 6: Defensive Hardening & Verification

We defensively hardened each function in train_fare.py against malformed inputs without breaking valid functionality:

- **parse_pnr:** Validates string type, strips whitespace, verifies exactly 10 digits; returns (zone, number) if valid, None otherwise.
- **journey_distance:** Validates string type, splits by comma, checks >= 2 entries, parses valid integers, ensures non-decreasing order; returns total distance or None.
- **berth_type:** Validates string type, splits on exactly one hyphen, validates non-empty coach and positive seat number; returns (coach, berth) or None.



4. Deliverables Checklist

- ✔ **Fixed train_fare.py**: Defensively hardened application script that resists all malformed inputs.
- ✔ **Updated test_train_fare.py**: Completed Hypothesis property-based fuzz tests.
- ✔ **Terminal Output Screenshots**: Step 2, Step 4, Step 5, and Step 6 saved under Lab 7/screenshots/.
- ✔ **PDF Document with Screenshots**: Full test and hardening verification report.